Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

Información general

Fecha de creación	24 – 09 - 2025	**Dirección de operaciones**

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Principios Avanzados de Programación Orientada a Objetos

Introducción

En el mundo del desarrollo de software, crear un programa que funcione es solo el primer paso, el punto de partida. La verdadera maestría, aquella que distingue a un programador competente de un verdadero arquitecto de software, reside en la capacidad de construir soluciones que no solo resuelven un problema hoy, sino que también son capaces de crecer, adaptarse y evolucionar con el tiempo. Esto es, en esencia, escribir buen software.

Pero ¿qué hace que un software sea bueno? No se trata solo de que no tenga errores. Se trata de su calidad interna: su claridad, su robustez y su mantenibilidad. Aquí es donde entran en juego los principios avanzados de la Programación Orientada a Objetos (POO). Conceptos como la alta cohesión, el bajo acoplamiento y un manejo de excepciones estratégico son los pilares sobre los que se construyen las aplicaciones más exitosas del mercado, desde las aplicaciones bancarias que usamos a diario en Colombia hasta las plataformas de Streaming con millones de usuarios.

Ignorar estos principios es como construir un edificio con cimientos débiles: puede que se mantenga en pie al principio, pero añadir un nuevo piso o reparar una grieta se convierte en una tarea titánica y peligrosa. En este documento, no solo aprenderás a escribir código, sino a diseñarlo con intención y visión a futuro.
Aprenderás a construir software que no solo funcione hoy, sino que sea fácil de evolucionar mañana.

Objetivos

El propósito de este material es llevar tus habilidades de programación más allá de la simple implementación de funcionalidades, para que puedas justificar tus decisiones de diseño con argumentos técnicos sólidos.
Al finalizar el estudio de este documento, estarás en capacidad de:

- Identificar los conceptos de cohesión, acoplamiento y reutilización en fragmentos de código Java existentes, reconociendo "mal código " que afectan la calidad.
- Aplicar de manera efectiva las estructuras de manejo de excepciones (try-catch-finally) para construir programas robustos que anticipen y gestionen fallos de manera controlada.
- Analizar el impacto directo que un diseño con alto acoplamiento y baja cohesión tiene sobre la dificultad para mantener y escalar una aplicación, utilizando métricas simples para evaluar la calidad del código.
- Evaluar y proponer mejoras concretas a un diseño de software existente, refactorizando el código para aplicar los principios avanzados de la POO y justificando el impacto positivo de dichos cambios.

Fundamentos teóricos

Todo software, desde el más simple hasta el más complejo, posee una estructura interna, una especie de esqueleto invisible para el usuario final. La calidad de esta estructura es lo que determina si una aplicación será robusta y adaptable o frágil y costosa de mantener. En este capítulo, exploraremos los principios arquitectónicos que forman los cimientos del software de calidad, no como reglas abstractas, sino como herramientas prácticas para tomar mejores decisiones de diseño.

1.1. Cohesión: El arte de hacer una sola cosa y hacerla bien

La **cohesión** mide qué tan enfocadas están las responsabilidades de un módulo de software, como una clase. Una clase con **alta cohesión** tiene una única misión, bien definida, y todos sus componentes (métodos y atributos) trabajan en conjunto para cumplirla. Por el contrario, una clase con **baja cohesión** intenta hacer demasiadas cosas no relacionadas entre sí, convirtiéndose en un "cajón de sastre" difícil de entender y modificar. Como señalan diversos autores, el objetivo es diseñar componentes que tengan responsabilidades claras y específicas (Barnes & Kölling, 2017). Pensemos en una analogía culinaria:

- **Alta Cohesión:** Un cuchillo de chef. Es una herramienta diseñada con un propósito claro: cortar. Todas sus características (el filo, el peso, el mango) contribuyen a esa única función. Es predecible y eficiente.
- **Baja Cohesión:** Una navaja suiza multifuncional. Aunque útil en ciertas situaciones, si solo queremos cortar verduras, tendremos que lidiar con el peso y la incomodidad de la lupa, las tijeras y el destornillador que no necesitamos. Si una de sus herramientas se daña, afecta a todo el conjunto.

En POO, una clase Factura que solo se encarga de calcular totales, añadir ítems y aplicar impuestos es altamente cohesiva. Si esa misma clase también se encargara de enviar correos electrónicos, conectarse a la base de datos y validar usuarios, su cohesión sería muy baja, volviéndola frágil y confusa (Moreno Pérez, 2015).

1.2. Acoplamiento: Los lazos que unen o ahogan tu código

El **acoplamiento** se refiere al grado de interdependencia entre dos o más módulos de software. Un **bajo acoplamiento** (o acoplamiento débil) significa que las clases son independientes entre sí; un cambio en una clase no debería obligar a realizar cambios en otras. Por otro lado, un **alto acoplamiento** (o acoplamiento fuerte) crea una red de dependencias frágiles, donde modificar un componente puede generar una cascada de errores inesperados en todo el sistema. El ideal es "minimizar el acoplamiento entre objetos para reducir el impacto de los cambios" (Vélez Serrano et al., 2011, p. 54). Usemos una analogía de la vida cotidiana:

- **Bajo Acoplamiento:** La relación entre tu televisor y el enchufe de la pared. El televisor solo necesita saber que existe una fuente de energía estándar. Puedes cambiar de televisor (un LG por un Samsung) sin necesidad de llamar a un electricista para que modifique el enchufe. La interfaz (el tomacorriente) es universal y estable.
- **Alto Acoplamiento:** Un celular antiguo con un cargador de diseño exclusivo. Si pierdes o se daña ese cargador específico, el celular se vuelve inútil. El teléfono depende *directamente* de esa implementación particular del cargador.

En el software, si una clase Reporte necesita acceder directamente a las variables internas de la clase Usuario para funcionar, el acoplamiento es alto. La solución de bajo acoplamiento sería que la clase Usuario ofrezca un método público como getDatosParaReporte(), ocultando su funcionamiento interno y proporcionando una interfaz estable (López Goytia, 2015).

A continuación, la **Figura 1** ilustra esta diferencia.

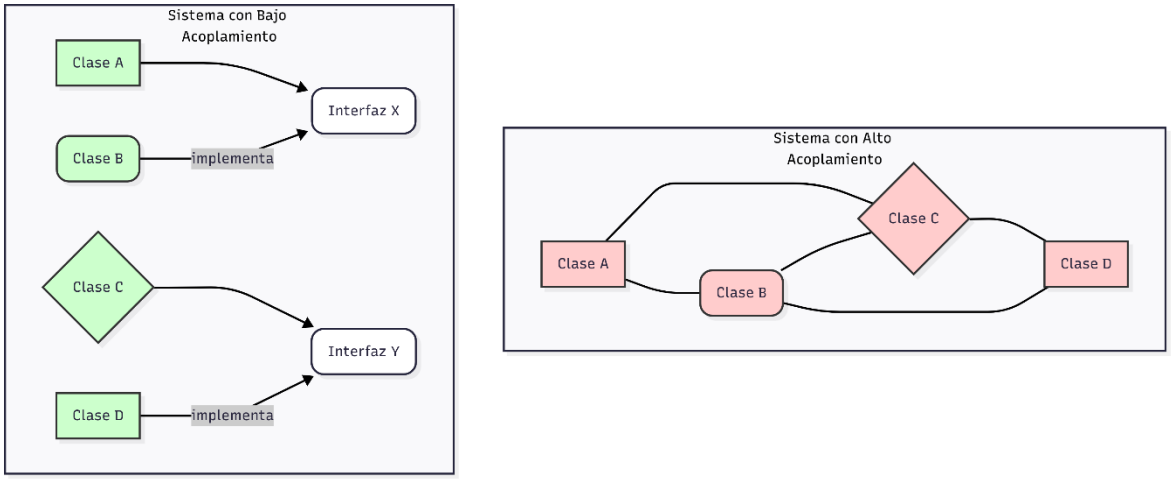

Figura 1. *Comparación visual entre acoplamiento alto y bajo.*

El diagrama muestra a la derecha cuatro clases enredadas con múltiples conexiones directas, representando alto acoplamiento. A la izquierda, las clases se comunican a través de interfaces, representando bajo acoplamiento.

1.3. Reutilización de Código: No reinventes la rueda, mejórala

Uno de los grandes beneficios de la POO es la capacidad de reutilizar código, lo que ahorra tiempo, reduce errores y aumenta la productividad. La reutilización no es simplemente "copiar y pegar"; es un principio de diseño que se logra principalmente a través de dos mecanismos: la **herencia** y la **composición**. Como lo explica Ceballos Sierra (2018), la herencia permite que una clase (subclase) adquiera los atributos y métodos de otra (superclase), mientras que la composición permite construir objetos complejos a partir de otros más simples ("un objeto *tiene* otro objeto").

La elección entre herencia y composición es una decisión de diseño crucial. La herencia crea un acoplamiento fuerte entre la superclase y la subclase, mientras que la composición favorece la flexibilidad y el bajo acoplamiento. Un buen diseñador sabe cuándo especializar (herencia) y cuándo delegar (composición).

1.4. Manejo de Excepciones: La red de seguridad de tu código

Un programa interactúa constantemente con elementos fuera de su control: la red puede caerse, un archivo puede no existir, o un usuario puede ingresar un dato inválido. Una **excepción** es un evento anómalo que ocurre durante la ejecución de un programa e interrumpe su flujo normal. El **manejo de excepciones** es el mecanismo que nos permite capturar estos eventos y responder a ellos de manera controlada, evitando que la aplicación se cierre abruptamente.
En lugar de dejar que el programa "muera", el código se envuelve en bloques especiales (try-catch en Java) que "intentan" ejecutar una operación riesgosa y "capturan" la excepción si algo sale mal, permitiéndonos mostrar un mensaje amigable al usuario, registrar el error o intentar una acción alternativa (Blasco, 2019). Un buen manejo de excepciones es un rasgo distintivo del software profesional y robusto, pues demuestra que la aplicación fue diseñada pensando no solo en el "camino feliz", sino también en los posibles problemas.

Aplicaciones en la vida cotidiana

Los principios teóricos del software adquieren su verdadero valor cuando los vemos funcionar en el mundo real. Cada vez que usamos una aplicación en nuestro celular, realizamos una compra en línea o interactuamos con un servicio digital, estamos siendo testigos del resultado de un buen (o mal) diseño de software. En este capítulo, vamos a "abrir el capó" de aplicaciones y sistemas que conocemos en Colombia para descubrir cómo la cohesión, el acoplamiento y el manejo de excepciones son los verdaderos motores de una experiencia digital de calidad.

2.1. La Arquitectura Invisible de las Super-Apps Colombianas

Pensemos en una aplicación de alta demanda como **Rappi**. Su éxito no solo radica en su idea de negocio, sino en su capacidad para ser un "todo en uno": restaurantes, supermercados, farmacias, favores, viajes. ¿Cómo es posible gestionar tal complejidad sin que el sistema colapse? La respuesta está en una arquitectura modular, un claro reflejo de la **alta cohesión** y el **bajo acoplamiento**.

- **Cohesión en Acción (Micro-Responsabilidades):** Dentro de Rappi, cada función es un módulo altamente cohesivo. No hay una sola entidad "Rappi" que lo haga todo. En su lugar, tenemos servicios especializados:
    - **Servicio de Autenticación:** Su única misión es verificar quién eres (login, registro, recuperación de contraseña). No sabe ni le interesa si vas a pedir una pizza o un mercado.
    - **Servicio de Catalogo Restaurantes:** Se dedica exclusivamente a gestionar menús, precios y disponibilidad. No tiene idea de quién es el usuario ni cómo va a pagar.
    - **Servicio de Procesamiento Pagos:** Su única responsabilidad es comunicarse con las pasarelas de pago (Wompi, PayU) para procesar una transacción. Recibe un monto y una orden, nada más.
    - **Servicio de geolocalización:** Maneja mapas, calcula rutas y sigue la ubicación del repartidor en tiempo real.

Esta separación (alta cohesión) permite que equipos de desarrolladores trabajen en paralelo en diferentes módulos sin generar conflictos. Además, si el módulo de catálogos falla, los usuarios que ya están en proceso de pago no se ven afectados (Nadal, 2021).

- **Bajo Acoplamiento como Estrategia de Crecimiento:** El bajo acoplamiento es lo que le da a Rappi su "superpoder" de escalar. Cuando decidieron añadir "Rappi Travel", no tuvieron que modificar el motor de pagos ni el sistema de autenticación. Simplemente crearon un nuevo servicio de ReservasViajes y lo "enchufaron" a los servicios ya existentes. El servicio de pagos no necesitó un if (esViaje) { cobrarAsi; }

else if (esRestaurante) { cobrarAsa; }. Simplemente recibió una orden de cobro, indiferente a su origen. Esta independencia es la clave para una evolución rápida y segura del producto.

2.1.1. Las APIs: El Contrato del Bajo Acoplamiento

Una manifestación muy concreta del bajo acoplamiento son las **APIs (Interfaces de Programación de Aplicaciones)**. Pensemos en cualquier e-commerce en Colombia que permita pagar a través de **PSE**. El e-commerce no tiene acceso directo a la base de datos de tu banco. Sería un caos de seguridad y una pesadilla de acoplamiento.

En su lugar, PSE ofrece una API, que es un "contrato" o un conjunto de reglas de comunicación. El e-commerce simplemente le dice a la API: "El usuario X quiere pagar Y pesos desde el banco Z". La API de PSE se encarga del resto de forma segura y oculta la complejidad. El e-commerce y el banco están **débilmente acoplados** a través de este intermediario. Si el banco cambia su sistema interno mañana, mientras siga respetando el "contrato" de la API, el e-commerce no se verá afectado en lo más mínimo.

2.2. Algo salió mal: El Manejo de Excepciones que Salva el Negocio

Todos hemos visto mensajes de error, pero un manejo de excepciones profesional va más allá de evitar que la aplicación se cierre. Es una herramienta crucial para proteger la **integridad de los datos** y la **lógica del negocio**.

- **Excepciones Técnicas (Protegiendo al Usuario):**
    1. **Fallo de Red:** Estás a punto de confirmar tu compra en Mercado Libre y se cae la señal de internet. Una aplicación robusta captura la excepción de conexión (ConnectException) y hace dos cosas: primero, te informa ("No tienes conexión a internet") y segundo, y más importante, **revierte la transacción localmente**, asegurando que no quedes en un estado inconsistente donde crees que pagaste, pero el servidor nunca recibió la confirmación.

    2. **Archivo Inexistente:** Intentas descargar un extracto bancario en PDF desde la aplicación de Davivienda. El servidor, por alguna razón, no encuentra el archivo. En lugar de un error genérico, la aplicación captura una FileNotFoundException y presenta un mensaje específico: "El extracto solicitado no está disponible en este momento. Por favor, inténtelo más tarde o contacte a soporte".

- **Excepciones de Negocio (Protegiendo las Reglas):** Las excepciones también son una forma elegante de hacer cumplir las reglas del negocio. Imaginemos el código de un cajero automático:

```
public void retirarDinero(double monto) throws SaldoInsuficienteException {
   if (monto > this.saldoDisponible) {
      throw new SaldoInsuficienteException("No es posible retirar " + monto + ". Saldo
disponible: " + this.saldoDisponible);
   }
   this.saldoDisponible -= monto;
   //  dispensar billetes
}
```

Aquí, SaldoInsuficienteException no es un error técnico. Es una excepción **creada a medida** para representar un evento que, aunque posible, viola una regla fundamental del negocio. Como lo describe Vegas (2022) en sus trabajos sobre programación avanzada, el manejo de excepciones permite separar la lógica de negocio principal del manejo de casos anómalos, haciendo el código mucho más limpio y legible.

2.3. La Deuda Técnica: Historia de una Startup

Imaginemos a "TuEnvíoYa", una startup de logística fundada en Bogotá. En sus inicios, dos programadores construyeron la plataforma rápidamente para salir al mercado. Crearon una única y gigantesca clase llamada GestionEnvio que hacía de todo: calculaba tarifas, asignaba mensajeros, procesaba pagos, enviaba notificaciones por correo y actualizaba la base de datos. El código era un ejemplo de **baja cohesión** y **alto acoplamiento.**

A medida que el negocio crecía, los desarrolladores empezaron a sufrir los siguientes "dolores" o **"code smells" (malos olores de código)**:

- **El Efecto Dominó (Fragilidad):** Un día, el gobierno cambió el cálculo del IVA. El desarrollador modificó las 3 líneas de código correspondientes en el método calcularTarifa(). Al día siguiente, el sistema de notificaciones por correo electrónico dejó de funcionar. Nadie entendía por qué, hasta que descubrieron que una variable usada para formatear los correos se reutilizaba sin cuidado dentro del cálculo de tarifas. El sistema estaba tan acoplado que un cambio trivial generaba fallos en cascada.
- **Código Inmortal (Rigidez):** Querían ofrecer un nuevo servicio: "Envío Programado". Sin embargo, la lógica para asignar un mensajero estaba tan mezclada con la de los pagos y las notificaciones que era imposible reutilizarla. La única opción era copiar y pegar cientos de líneas de código, duplicando el problema.
- **Arqueología de Código (Complejidad):** Un nuevo desarrollador tardó una semana entera en entender cómo se procesaba un solo envío. La clase GestionEnvio tenía más de 5,000 líneas y docenas de variables globales. Modificarla era como

desactivar una bomba, había que seguir cada rastro de cada variable para asegurarse de que el cambio no explotara en otro lugar.

La refactorización no fue un lujo, sino una necesidad para la supervivencia del negocio. La lección fue clara: la "velocidad" inicial de un mal diseño es un espejismo que se paga con una parálisis casi total en el futuro.

Casos de estudio

La teoría y los ejemplos nos han dado el mapa, pero es en la resolución de problemas complejos donde se forja la verdadera pericia. En este capítulo, nos sumergiremos en un escenario profesional detallado que ilustra cómo la aplicación rigurosa de los principios de POO no es solo una buena práctica técnica, sino una necesidad estratégica para el éxito de un negocio.

3.1. El Contexto: "AgroTech-CUN" y su Crisis de Crecimiento

"AgroTech-CUN" es una startup ficticia con sede en Tocancipá que ofrece una plataforma de software para que los agricultores de la sabana de Bogotá gestionen sus cultivos. Su producto estrella es un "Módulo de Reportes de Cosecha" que analiza datos de siembra, clima y suelo para predecir el rendimiento y la rentabilidad.
Durante el primer año, la plataforma fue un éxito. Sin embargo, a medida que la base de clientes creció y solicitaron nuevas funcionalidades, el equipo de desarrollo se encontró con un muro.

3.2. El Desafío: Un Módulo "Monolítico" al Borde del Colapso

El problema residía en una única y monstruosa clase llamada ReporteManager.java. Esta clase de 8.000 líneas de código era responsable de todo el proceso, evidenciando graves fallas de diseño:

1. **Baja Cohesión Crítica:** La clase se conectaba directamente a la base de datos, ejecutaba complejos algoritmos de predicción, formateaba los resultados en PDF y Excel, y enviaba los reportes por correo electrónico. Era el ejemplo perfecto de una "God Class" o clase todopoderosa.
2. **Alto Acoplamiento Extremo:** Estaba fuertemente acoplada a la librería iText para PDF, a la librería Apache POI para Excel, al driver de la base de datos PostgreSQL y al servidor de correo. Cualquier actualización en una de estas librerías externas requería una revisión completa y riesgosa de toda la clase.
3. **Manejo de Excepciones Inexistente:** No había bloques try-catch. Si la conexión a la base de datos fallaba por un instante, todo el proceso de generación de reportes se abortaba y el programa se cerraba con un error críptico, dejando al agricultor sin su informe.

El **impacto en el negocio** era devastador:

- **Tiempo de Desarrollo Lento:** Añadir un nuevo campo al reporte (ej. "nivel de humedad") tomaba tres semanas de desarrollo y pruebas.
- **Inflexibilidad:** Los clientes pedían los reportes en formato CSV, pero implementarlo se consideraba "imposible" sin reescribir todo el módulo.

- **Pérdida de Clientes:** La inestabilidad y lentitud del módulo estaban generando una alta tasa de cancelación del servicio.

3.3. La Solución Aplicada: Refactorización Estratégica Basada en Principios POO

Un arquitecto de software fue contratado para liderar el rescate. En lugar de "apagar el incendio" con parches, propuso una refactorización completa, guiada por los principios de diseño. El proceso de "diseñar y programar" debía ser replanteado desde sus bases (Vélez Serrano et al., 2011).

Paso 1: Descomponer por Responsabilidades (Aumentar Cohesión)
La clase ReporteManager.java fue desmantelada. Se crearon varias clases nuevas, cada una con una única y clara responsabilidad (alta cohesión):

- RepositorioCosecha.java: Su única misión era interactuar con la base de datos para obtener los datos crudos.
- MotorDeCalculo.java: Contenía la lógica de negocio pura para procesar los datos y generar las predicciones. No sabía nada de bases de datos ni de formatos de archivo.
- ServicioDeNotificacion.java: Encargada exclusivamente de enviar correos electrónicos.

Paso 2: Invertir las Dependencias (Disminuir Acoplamiento)
Para el formateo, se aplicó el **Principio de Inversión de Dependencias**. Se creó una interfaz llamada IFormateadorReporte:

```
// Interfaz que define el "contrato" para cualquier formateador
public interface IFormateadorReporte {
   File formatear(ReporteData datos);
}
```

Luego, se crearon implementaciones concretas de esta interfaz:

- FormateadorPDF.java **implements** IFormateadorReporte
- FormateadorExcel.java **implements** IFormateadorReporte

El nuevo orquestador del proceso ya no dependía de las clases concretas, sino de la abstracción. Esto significa que para añadir el formato CSV, los desarrolladores solo tuvieron que crear una nueva clase FormateadorCSV.java que implementara la interfaz, sin tocar una sola línea del código existente. Esta técnica es fundamental para crear software flexible y extensible (Ceballos Sierra, 2018).

Paso 3: Implementar una Estrategia de Excepciones

Se introdujo un manejo de excepciones robusto. El RepositorioCosecha ahora podía lanzar una DataAccessLayerException si la base de datos fallaba, y cada formateador podía lanzar una FormattingException. El orquestador principal envolvía las llamadas en bloques try-catch, permitiendo un control granular de los errores: si fallaba la generación del PDF, el sistema podía intentar generar el de Excel como alternativa o, como mínimo, mostrar un mensaje claro al usuario sin abortar la aplicación.

3.4. El Impacto: Medición del Éxito Post-Refactorización

Los resultados de la refactorización se midieron con datos concretos, demostrando el valor de la inversión en calidad de software:

- **Métricas de Negocio:**
  - **Time-to-Market:** El tiempo para implementar un nuevo formato de reporte (como CSV) se redujo de "imposible" a **2 días**.
  - **Satisfacción del Cliente:** Las quejas por errores en el módulo de reportes disminuyeron en un **95%** en los siguientes tres meses.
  - **Retención:** La tasa de cancelación de clientes se redujo en un **40%**.
- **Métricas Técnicas:**
  - **Mantenibilidad:** El índice de complejidad ciclomática del código se redujo en un **70%**.
  - **Calidad:** La cobertura de pruebas unitarias pasó del 5% al **90%**, permitiendo hacer cambios con confianza.
  - **Rendimiento:** La generación de reportes se aceleró en un **30%** gracias a la optimización de las consultas a la base de datos, ahora aisladas en su propia clase.

Este caso demuestra que la cohesión, el acoplamiento y el manejo de excepciones no son conceptos académicos, sino herramientas vitales que tienen un impacto directo y medible en la agilidad, la calidad y la rentabilidad de un proyecto de software.

Herramientas y técnicas

Saber la teoría es importante, pero la verdadera maestría en el desarrollo de software se demuestra con el teclado. Este capítulo es eminentemente práctico. Te guiará a través de ejercicios concretos en Java, utilizando el IDE NetBeans, para que puedas implementar los principios de cohesión, acoplamiento y manejo de excepciones. Pasaremos de ser observadores a ser constructores de software de calidad.

4.1. Taller Práctico con Java y NetBeans

Abre tu entorno de desarrollo NetBeans y crea un nuevo proyecto de Java. Vamos a trabajar en dos ejercicios diseñados para solidificar los conceptos clave.

Ejercicio 1: Implementando una Red de Seguridad con try-catch-finally

Objetivo: Aprender a manejar excepciones de forma controlada para evitar que el programa se cierre inesperadamente al interactuar con archivos.

Escenario: Vamos a escribir un programa que intenta leer la primera línea de un archivo de texto llamado config.txt. Este archivo podría no existir, lo que provocaría un error si no lo manejamos adecuadamente.

Instrucciones:

1. Crea una nueva clase LectorArchivos.java.
2. Copia y pega el siguiente código en tu clase:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LectorArchivos {

    public void leerConfiguracion() {
        BufferedReader reader = null; // Declaramos el lector fuera del try
        try {
            // Paso 1: Intentamos abrir el archivo. Esta línea puede lanzar una IOException.
            System.out.println("Intentando leer el archivo 'config.txt'...");
            reader = new BufferedReader(new FileReader("config.txt"));
            String primeraLinea = reader.readLine();

            if (primeraLinea != null) {
                System.out.println("Éxito. La primera línea es: " + primeraLinea);
```

```
            } else {
                System.out.println("El archivo está vacío.");
            }

        } catch (IOException e) {
            // Paso 2: Si ocurre un error (ej. el archivo no existe), se captura aquí.
            System.err.println("Error: No se pudo encontrar o leer el archivo 'config.txt'.");
            System.err.println("Detalle técnico: " + e.getMessage());

        } finally {
            // Paso 3: Este bloque se ejecuta SIEMPRE, haya o no un error.
            // Es el lugar ideal para cerrar recursos y evitar fugas de memoria.
            System.out.println("Bloque 'finally': intentando cerrar el archivo.");
            if (reader != null) {
                try {
                    reader.close();
                } catch (IOException e) {
                    System.err.println("Error al intentar cerrar el lector de archivos.");
                }
            }
        }
    }
}
```

Para probarlo: Ejecuta el método leerConfiguracion.
Primero, sin crear el archivo config.txt en la raíz de tu proyecto; verás cómo el bloque catch se activa. Luego, crea el archivo, escribe algo en él y vuelve a ejecutarlo; verás el flujo exitoso. Este ejercicio demuestra cómo un programa robusto se anticipa a los problemas (Flórez Fernández, 2012).

Ejercicio 2: Refactorizando una "God Class"

Objetivo: Aplicar los principios de alta cohesión y bajo acoplamiento para mejorar un diseño de software deficiente.

Escenario: Se te entrega la siguiente clase GestionTodo.java, que gestiona usuarios, tareas y las guarda en un archivo. Es un ejemplo clásico de baja cohesión.

```
// CÓDIGO CON "MALOS OLORES" - ANTES DE REFACTORIZAR
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
```

```
import java.util.List;

public class GestionTodo {
    private List<String> usuarios = new ArrayList<>();
    private List<String> tareas = new ArrayList<>();

    public void agregarUsuario(String nombre) {
        usuarios.add(nombre);
        System.out.println("Usuario agregado: " + nombre);
    }

    public void agregarTarea(String descripcion) {
        tareas.add(descripcion);
        System.out.println("Tarea agregada: " + descripcion);
    }

    // Esta clase también sabe cómo guardar en disco, ¡rompiendo la cohesión!
    public void guardarEnArchivo() {
        try (FileWriter writer = new FileWriter("datos.txt")) {
            writer.write("--- USUARIOS ---\n");
            for (String u : usuarios) {
                writer.write(u + "\n");
            }
            writer.write("--- TAREAS ---\n");
            for (String t : tareas) {
                writer.write(t + "\n");
            }
            System.out.println("Datos guardados en archivo.");
        } catch (IOException e) {
            System.err.println("Error al guardar el archivo: " + e.getMessage());
        }
    }
}
```

Tu Misión: Refactoriza el código anterior dividiendo la clase GestionTodo en **tres clases más cohesivas y débilmente acopladas**.

1. GestionUsuario.java: Responsable únicamente de la lógica de usuarios.
2. GestionTarea.java: Responsable únicamente de la lógica de tareas.
3. PersistenciaDatos.java: Responsable únicamente de guardar los datos en un archivo.

Justificación: Al separar las responsabilidades, logras que el sistema sea más fácil de mantener y extender. Mañana, si necesitas cambiar el formato de guardado de archivo a una base de datos, solo tendrás que modificar PersistenciaDatos.java, sin tocar la lógica de usuarios o tareas. Este es el poder de un buen diseño orientado a objetos (Barnes & Kölling, 2017).

4.2. Visualizando el Diseño: Más Allá del Código

Aunque el código es la implementación final, las herramientas de modelado nos ayudan a pensar y discutir el diseño antes de escribir una sola línea. Herramientas como **UML (Lenguaje Unificado de Modelado)** permiten crear diagramas de clases que visualizan las relaciones, dependencias y responsabilidades en un sistema.

Antes de realizar la refactorización del Ejercicio 2, por ejemplo, un buen profesional dibujaría un diagrama de clases de la solución propuesta. Esto ayuda a identificar problemas de acoplamiento y a validar la cohesión de cada componente. Como afirma Vélez Serrano et al. (2011), usar UML y Java en conjunto es una práctica que fortalece enormemente el proceso de diseño y programación.

Incluso sin software especializado, puedes usar herramientas en línea como draw.io o simplemente papel y lápiz para esbozar tus clases, sus atributos y métodos, y las flechas que indican sus relaciones. Esta práctica de visualización es invaluable para pasar de programador a arquitecto de software.

Conclusión

A lo largo de este documento, hemos viajado desde los fundamentos teóricos del buen diseño de software hasta su aplicación en el mundo real y profesional. Hemos desglosado conceptos que a primera vista pueden parecer abstractos —**alta cohesión, bajo acoplamiento y manejo de excepciones**— para descubrir que son, en realidad, las herramientas más prácticas y poderosas en el arsenal de un desarrollador de software.

La lección más importante no es memorizar una definición, sino comprender la filosofía que subyace a estos principios: construir software no es solo un acto técnico, sino un ejercicio de **previsión, organización y profesionalismo**. Escribir código que "funciona" es el requisito mínimo; el verdadero objetivo es crear soluciones que sean mantenibles, escalables y robustas. Como vimos en el caso de "AgroTech-CUN", la deuda técnica generada por un mal diseño no es un problema abstracto, sino un lastre que frena la innovación y pone en riesgo el negocio.

La Programación Orientada a Objetos, en su nivel avanzado, te invita a pensar más como un arquitecto y menos como un simple constructor. Se trata de preguntarte constantemente: ¿Cuál es la responsabilidad única de esta clase? ¿Qué tan dependiente es este módulo de los demás? ¿Qué podría salir mal aquí y cómo puedo manejarlo de forma elegante?

Al cerrar este capítulo, te invito a llevar esta mentalidad a cada línea de código que escribas. No veas estos principios como reglas restrictivas, sino como un camino hacia la maestría en tu oficio. El software que construyas hoy será el legado con el que otros desarrolladores (o incluso tu "yo" del futuro) tendrán que trabajar mañana. Asegúrate de que sea un legado del que te sientas orgulloso.

Para continuar tu reflexión, te dejo con estas preguntas abiertas:

1. De los "malos olores de código" que vimos (como la "God Class"), ¿cuál crees que es el más común en los proyectos en los que has participado y por qué crees que ocurre con tanta frecuencia?
2. Más allá de try-catch, ¿qué otras estrategias o prácticas podrías investigar para crear aplicaciones aún más resilientes y a prueba de fallos en tus futuros proyectos?

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

Referencias.

Aristizábal Martínez, D. A., & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos* (1ª ed.). Instituto Tecnológico Metropolitano.

Barnes, D. J., & Kölling, M. (2017). *Programación orientada a objetos con Java usando BlueJ*. Pearson Educación.

Blasco, F. (2019). *Programación Orientada a Objetos en JAVA*. RA-MA Editorial.
Ceballos Sierra, F. J. (2018). *Programación orientada a objetos con C++* (5ª ed.). RA-MA Editorial.

Flórez Fernández, H. A. (2012). *Programación orientada a objetos usando java*. Ecoe Ediciones.

López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*. Grupo Editorial Patria.

Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial.
Nadal, M. (2021). *Curso de programación Java* (1ª ed.). Difusora Larousse - Anaya Multimedia.

Vegas, J. (2022). *Java 17 Programación Avanzada*. RA-MA Editorial.

Vélez Serrano, J. F., Peña, A., & Giraldo, W. (2011). *Diseñar y programar, todo es empezar: Una introducción a la programación orientada a objetos usando UML y Java*. Dykinson.